INTERVIEW GUIDE · LINUX KERNEL & DRIVERS

Top 50 Linux Device Driver Interview Questions & Answers (2026)

By EmbeddedShiksha Team · Linux Kernel & Driver Development · April 2026

Character Drivers · Platform Drivers · DMA · Kernel Modules · Interrupts · Linux Interview

Linux device driver interviews are among the most challenging in the embedded and systems programming world. Companies like Qualcomm, Texas Instruments, NXP, NVIDIA, Bosch, and Continental routinely test deep knowledge of the kernel subsystem, driver models, memory management, DMA, and interrupt handling.

This guide covers the **top 50 Linux device driver interview questions** asked across all experience levels — freshers to 10+ year engineers — with precise answers and real kernel code examples. Topics include character drivers, platform drivers, DMA, interrupt handling, synchronization, and more.

Who this is for: Embedded Linux engineers, kernel developers, and anyone preparing for a Linux driver or BSP role. Questions are drawn from real interview experiences at top semiconductor and automotive companies.

Contents

1. Section 1: Driver Fundamentals & Kernel Basics (Q1–Q8)
2. Section 2: Character Drivers (Q9–Q18)
3. Section 3: Platform Drivers & Device Tree (Q19–Q28)
4. Section 4: DMA — Direct Memory Access (Q29–Q38)
5. Section 5: Interrupts & Bottom Halves (Q39–Q44)
6. Section 6: Synchronization & Memory (Q45–Q50)

Section 1: Driver Fundamentals & Kernel Basics

These questions test your understanding of how Linux organises drivers, the difference between kernel and user space, and core kernel APIs you will use in every driver you write.

Q1

What is a Linux device driver and what is its primary role?

A Linux device driver is a piece of kernel code that acts as an interface between the operating system and a hardware device. Its primary role is to abstract hardware complexity so that user-space applications can interact with hardware through standard system calls like `open()`, `read()`, `write()`, and `ioctl()` without knowing hardware details.

Drivers run in kernel space and have direct access to hardware registers, memory-mapped I/O, interrupts, and DMA. Linux classifies drivers into three main categories: **character drivers**, **block drivers**, and **network drivers**.

Q2

What is the difference between a kernel module and a built-in driver?

A **kernel module** (loadable module, `.ko` file) is compiled separately from the kernel and can be inserted or removed at runtime using `insmod`/`modprobe` and `rmmod`. A **built-in driver** is compiled directly into the kernel image (`vmlinux`) and is always present in memory.

Modules are selected with `CONFIG_FOO=m` in the kernel config; built-ins with `CONFIG_FOO=y`. Modules are preferred during development because they can be loaded and unloaded without rebooting the system.

Q3

Explain the purpose ofmodule_init()andmodule_exit().

`module_init(func)` registers `func` as the module's initialisation routine. It is called when the module is loaded via `insmod` or at boot time for built-ins. `module_exit(func)` registers the cleanup routine, called when the module is removed with `rmmod`.

```
#include <linux/module.h>
#include <linux/init.h>

static int __init my_driver_init(void)
{
    pr_info("my_driver: loaded\n");
    return 0; /* 0 = success; negative errno on failure */
}

static void __exit my_driver_exit(void)
{
    pr_info("my_driver: unloaded\n");
}
```

```
module_init(my_driver_init);
module_exit(my_driver_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("EmbeddedShiksha");
MODULE_DESCRIPTION("Demo driver");
```

The `__init` and `__exit` macros mark functions so the kernel can free their memory after use.

Q4

What is a major and minor number? How are they used?

Every device file in Linux has two numbers associated with it. The **major number** identifies the driver responsible for handling the device. The **minor number** identifies a specific instance of the device managed by that driver.

For example, `/dev/ttyS0` and `/dev/ttyS1` share the same major number (the serial driver) but have minor numbers 0 and 1 respectively. You can see these with `ls -l /dev`.

```
dev_t devno = MKDEV(major, minor); /* pack into dev_t */
int maj = MAJOR(devno);            /* extract major    */
int min = MINOR(devno);            /* extract minor    */
```

Q5

What is the difference betweenkmalloc()andvmalloc()?

`kmalloc()` allocates **physically contiguous** memory in kernel space and is suitable for DMA buffers and small allocations. It returns memory from the slab allocator and is fast. Size is limited (typically up to 4MB on most architectures).

`vmalloc()` allocates **virtually contiguous** but not necessarily physically contiguous memory. It uses page tables to map pages that may be scattered in physical memory. It is used for large allocations where physical contiguity is not needed.

> Use **kmalloc** when the memory will be used for DMA or hardware access. Use **vmalloc** when you need a large software buffer and don't care about physical layout.

Q6

What doescopy_to_user()andcopy_from_user()do and why are they necessary?

These functions safely transfer data between kernel space and user space. You cannot use a plain `memcpy()` because user-space pointers are not directly accessible from the kernel — the page may not be present or the pointer could be invalid (potential security vulnerability or kernel oops).

```
/* In driver read() — send data to user */
```

```
if (copy_to_user(user_buf, kernel_buf, count))
    return -EFAULT;

/* In driver write() — receive data from user */
if (copy_from_user(kernel_buf, user_buf, count))
    return -EFAULT;
```

Both functions return 0 on success, or the number of bytes that could *not* be copied on failure. Always return `-EFAULT` if they fail.

Q7

What isstruct file_operations?

`struct file_operations` (defined in `<linux/fs.h>`) is a table of function pointers that tells the kernel which driver functions to call when a user-space application performs system calls on a device file. It is the core interface of a character driver.

```
static struct file_operations my_fops = {
    .owner   = THIS_MODULE,
    .open    = my_open,
    .release = my_release,
    .read    = my_read,
    .write   = my_write,
    .unlocked_ioctl = my_ioctl,
    .llseek  = my_llseek,
};
```

Any function pointer left as NULL causes the kernel to use a default behaviour (e.g. `llseek` defaults to `default_llseek`).

Q8

What is the purpose ofTHIS_MODULEin a driver?

`THIS_MODULE` is a macro that expands to a pointer to the `struct module` representing the current kernel module. It is used in `struct file_operations` and other kernel structures to let the kernel track which module owns a resource. When set in `file_operations.owner`, the kernel prevents the module from being unloaded while a file descriptor to it is open, preventing use-after-free bugs.

Section 2: Character Drivers

Character drivers are the most common driver type. They handle devices that transfer data as a stream of bytes — serial ports, sensors, GPIOs, custom hardware accelerators.

Q9

Walk me through the steps to write a minimal character driver.

A minimal character driver requires these steps:

1. **Allocate a device number** (major + minor) using `alloc_chrdev_region()`.
2. **Initialise a** `cdev` with `cdev_init()`, linking it to your `file_operations`.
3. **Add the cdev to the kernel** with `cdev_add()`.
4. **Create a device class and device node** under `/dev` using `class_create()` and `device_create()`.
5. **In** `module_exit`, reverse every step: `device_destroy()`, `class_destroy()`, `cdev_del()`, `unregister_chrdev_region()`.

```
#include <linux/module.h>
#include <linux/cdev.h>
#include <linux/fs.h>
#include <linux/device.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "mychardev"
#define CLASS_NAME  "myclass"

static dev_t   devno;
static struct  cdev    my_cdev;
static struct  class  *my_class;
static struct  device *my_device;

static int my_open(struct inode *inode, struct file *file)
{
    pr_info("mychardev: open\n");
    return 0;
}

static ssize_t my_read(struct file *file, char __user *buf,
                       size_t len, loff_t *off)
{
    char msg[] = "Hello from kernel!\n";
    int  msg_len = sizeof(msg) - 1;
    if (*off >= msg_len) return 0;
    if (copy_to_user(buf, msg, msg_len)) return -EFAULT;
    *off += msg_len;
    return msg_len;
}

static struct file_operations fops = {
```

```
    .owner = THIS_MODULE,
    .open  = my_open,
    .read  = my_read,
};

static int __init mydev_init(void)
{
    alloc_chrdev_region(&devno, 0, 1, DEVICE_NAME);
    cdev_init(&my_cdev, &fops);
    my_cdev.owner = THIS_MODULE;
    cdev_add(&my_cdev, devno, 1);
    my_class  = class_create(THIS_MODULE, CLASS_NAME);
    my_device = device_create(my_class, NULL, devno, NULL, DEVICE_NAME);
    pr_info("mychardev: registered, major=%d\n", MAJOR(devno));
    return 0;
}

static void __exit mydev_exit(void)
{
    device_destroy(my_class, devno);
    class_destroy(my_class);
    cdev_del(&my_cdev);
    unregister_chrdev_region(devno, 1);
}

module_init(mydev_init);
module_exit(mydev_exit);
MODULE_LICENSE("GPL");
```

Q10

What is the difference betweenalloc_chrdev_region()andregister_chrdev_region()?

`register_chrdev_region()` lets you request a *specific* major number that you choose. If that number is already taken, registration fails. `alloc_chrdev_region()` asks the kernel to *dynamically assign* a free major number — this is the recommended approach because it avoids hard-coded conflicts.

```
/* Dynamic — preferred */
alloc_chrdev_region(&devno, first_minor, count, "mydev");

/* Static — if you need a specific major */
register_chrdev_region(MKDEV(240, 0), 1, "mydev");
```

Always prefer `alloc_chrdev_region()` in production drivers.

Q11

What isioctl()and how do you implement it in a driver?

`ioctl` (Input/Output Control) is a system call that lets user-space send device-specific commands and data to a driver — things that don't fit neatly into read/write. Commands are defined using macros like `_IO`, `_IOR`, `_IOW`, `_IOWR`.

```
/* Define commands in a header shared by driver and user app */
#define MY_MAGIC   'k'
#define MY_RESET   _IO(MY_MAGIC,  0)
#define MY_GET_VAL _IOR(MY_MAGIC, 1, int)
#define MY_SET_VAL _IOW(MY_MAGIC, 2, int)

/* In the driver */
static long my_ioctl(struct file *file,
                     unsigned int cmd, unsigned long arg)
{
    int val;
    switch (cmd) {
    case MY_RESET:
        /* reset hardware */
        break;
    case MY_GET_VAL:
        val = read_hw_register();
        if (copy_to_user((int __user *)arg, &val, sizeof(val)))
            return -EFAULT;
        break;
    case MY_SET_VAL:
        if (copy_from_user(&val, (int __user *)arg, sizeof(val)))
            return -EFAULT;
        write_hw_register(val);
        break;
    default:
        return -ENOTTY; /* unknown command */
    }
    return 0;
}
```

Q12

What ismmap()in a device driver and when would you use it?

`mmap()` maps device memory (or kernel buffers) directly into a user-space process's virtual address space, allowing the application to access hardware memory without system call overhead on every access. It is widely used in frame buffer drivers, GPU drivers, and high-speed data acquisition drivers.

```
static int my_mmap(struct file *file, struct vm_area_struct *vma)
{
```

```
    unsigned long phys_addr = MY_DEVICE_BASE_ADDR;
    unsigned long size = vma->vm_end - vma->vm_start;

    vma->vm_page_prot = pgprot_noncached(vma->vm_page_prot);

    if (remap_pfn_range(vma,
                        vma->vm_start,
                        phys_addr >> PAGE_SHIFT,
                        size,
                        vma->vm_page_prot))
        return -EAGAIN;
    return 0;
}
```

Q13

How doespoll()/select()work in a character driver?

To support `poll()` and `select()` system calls, the driver implements the `.poll` file operation. The driver must maintain a wait queue and wake it up when data becomes available (or the device is ready to accept data).

```
static DECLARE_WAIT_QUEUE_HEAD(my_wq);
static int data_ready = 0;

static __poll_t my_poll(struct file *file,
                        struct poll_table_struct *wait)
{
    __poll_t mask = 0;
    poll_wait(file, &my_wq, wait);
    if (data_ready)
        mask |= EPOLLIN | EPOLLRDNORM; /* readable */
    return mask;
}

/* In interrupt handler or producer: */
data_ready = 1;
wake_up_interruptible(&my_wq);
```

Q14

What is theprivate_datafield instruct fileused for?

`file->private_data` is a void pointer reserved for driver use. It is typically set in the driver's `open()` function to store a pointer to a per-device structure, so all other file operations can retrieve device state without global variables. This is essential when multiple instances of a device are open simultaneously.

```
struct my_dev {
    struct cdev cdev;
    int         value;
    /* ... */
};

static int my_open(struct inode *inode, struct file *file)
{
    struct my_dev *dev;
    dev = container_of(inode->i_cdev, struct my_dev, cdev);
    file->private_data = dev; /* store for later use */
    return 0;
}

static ssize_t my_read(struct file *file, char __user *buf,
                       size_t len, loff_t *off)
{
    struct my_dev *dev = file->private_data;
    /* use dev->value etc. */
}
```

Q15

What iscontainer_of()and how is it used in drivers?

`container_of(ptr, type, member)` is a macro that, given a pointer to a member of a structure, returns a pointer to the enclosing structure. It is the standard way in the Linux kernel to get a pointer to the outer device structure from an embedded sub-structure pointer.

```
/* If you have a pointer to the embedded cdev: */
struct my_dev *dev = container_of(inode->i_cdev,
                                  struct my_dev,
                                  cdev);
/* Now 'dev' points to the full my_dev structure */
```

It works by computing the byte offset of `member` within `type` and subtracting it from `ptr`.

Q16

What happens if a driver'sopen()returns a negative value?

If `open()` returns a negative value (a negated errno constant such as `-ENOMEM`, `-EBUSY`, or `-ENODEV`), the kernel translates it to a positive errno and sets it in the calling process's `errno` variable. The `open()` system call in user space returns `-1`. The file descriptor is not created. Always return a negative errno on failure, never a positive non-zero value.

Q17

How do you implement a circular buffer (ring buffer) in a character driver?

A ring buffer is common in UART/serial character drivers. The key idea is two indices — `head` (write pointer) and `tail` (read pointer) — wrapping around a fixed-size buffer.

```
#define BUF_SIZE 1024

struct ring_buf {
    char     buf[BUF_SIZE];
    unsigned head; /* write index */
    unsigned tail; /* read index  */
    spinlock_t lock;
};

static void rb_write(struct ring_buf *rb, char c)
{
    spin_lock(&rb->lock);
    rb->buf[rb->head % BUF_SIZE] = c;
    rb->head++;
    spin_unlock(&rb->lock);
}

static int rb_read(struct ring_buf *rb, char *c)
{
    spin_lock(&rb->lock);
    if (rb->head == rb->tail) {
        spin_unlock(&rb->lock);
        return -EAGAIN; /* empty */
    }
    *c = rb->buf[rb->tail % BUF_SIZE];
    rb->tail++;
    spin_unlock(&rb->lock);
    return 0;
}
```

> The kernel provides a ready-made ring buffer — **kfifo** — in `<linux/kfifo.h>`. Use it instead of rolling your own in production drivers.

Q18

What is the difference betweenO_NONBLOCKand blocking I/O in a driver?

In **blocking I/O** (the default), a `read()` call on an empty device puts the process to sleep until data is available. In **non-blocking I/O**, the same call returns immediately with `-EAGAIN` if no data is ready. The driver checks the flag with `file->f_flags & O_NONBLOCK`.

```
static ssize_t my_read(struct file *file, char __user *buf,
                       size_t len, loff_t *off)
{
    if (!data_available) {
        if (file->f_flags & O_NONBLOCK)
            return -EAGAIN;
        /* blocking: sleep until data arrives */
        if (wait_event_interruptible(my_wq, data_available))
            return -ERESTARTSYS;
    }
    /* copy data to user */
    return bytes_copied;
}
```

Section 3: Platform Drivers & Device Tree

Platform drivers are how most SoC peripheral drivers are written. They integrate with the Linux device model and Device Tree to describe hardware topology.

Q19

What is a platform driver and how does it differ from a PCI or USB driver?

A **platform driver** handles devices that are hardwired onto an SoC or board and cannot be discovered automatically at runtime (e.g., UARTs, I2C controllers, SPI controllers, GPIO banks). Their existence and configuration are described statically in the **Device Tree** (or legacy board files).

PCI/USB drivers use their respective bus's enumeration protocol to discover devices dynamically. Platform drivers use a *name-matching* or *compatible-string matching* mechanism — the kernel matches the device tree `compatible` property against the driver's `of_match_table`.

Q20

Write a minimal platform driver skeleton.

```
#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/of.h>

struct my_priv {
    void __iomem *base;
    int           irq;
};

static int my_probe(struct platform_device *pdev)
{
    struct my_priv    *priv;
    struct resource   *res;

    priv = devm_kzalloc(&pdev->dev, sizeof(*priv), GFP_KERNEL);
    if (!priv)
        return -ENOMEM;

    /* Get memory-mapped I/O region from device tree */
    res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
    priv->base = devm_ioremap_resource(&pdev->dev, res);
    if (IS_ERR(priv->base))
        return PTR_ERR(priv->base);

    /* Get IRQ from device tree */
    priv->irq = platform_get_irq(pdev, 0);
    if (priv->irq < 0)
        return priv->irq;
```

```
    platform_set_drvdata(pdev, priv);
    dev_info(&pdev->dev, "probed successfully\n");
    return 0;
}

static int my_remove(struct platform_device *pdev)
{
    dev_info(&pdev->dev, "removed\n");
    return 0;
}

static const struct of_device_id my_of_match[] = {
    { .compatible = "myvendor,mydevice" },
    { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, my_of_match);

static struct platform_driver my_driver = {
    .probe  = my_probe,
    .remove = my_remove,
    .driver = {
        .name           = "mydevice",
        .of_match_table = my_of_match,
    },
};
module_platform_driver(my_driver);

MODULE_LICENSE("GPL");
```

Q21

What isdevm_*(devres)? Why should you use it?

`devm_*` functions are **device-managed resources**. Any resource allocated with a `devm_` variant (e.g., `devm_kzalloc`, `devm_ioremap_resource`, `devm_request_irq`) is automatically freed by the kernel when the device is detached or the driver is unloaded — even if `probe()` fails partway through.

This eliminates complex cleanup code in `remove()` and error paths, significantly reducing resource leak bugs.

> In modern kernel drivers, almost all resource allocation is done with **devm_** variants. If you're writing a new driver without `devm_`, reviewers will ask why.

Q22

What is the Device Tree and what role does it play in platform drivers?

The **Device Tree (DT)** is a data structure that describes the hardware topology of a board to the Linux kernel. It is written in Device Tree Source (`.dts`) files and compiled to a Device Tree Blob (`.dtb`) by the Device Tree Compiler (`dtc`).

A device tree node for a platform device looks like:

```
mydevice@40010000 {
    compatible = "myvendor,mydevice";
    reg = <0x40010000 0x1000>; /* base address, size */
    interrupts = <0 42 4>;     /* GIC SPI 42, level high */
    clocks = <&apb_clk>;
    status = "okay";
};
```

The kernel matches the `compatible` string to the driver's `of_match_table`, then calls `probe()` with the populated `platform_device`.

Q23

How do you read a property from the Device Tree in a driver?

```
#include <linux/of.h>
#include <linux/of_device.h>

static int my_probe(struct platform_device *pdev)
{
    struct device_node *np = pdev->dev.of_node;
    u32 speed;
    const char *name;

    /* Read a u32 property */
    if (of_property_read_u32(np, "max-speed", &speed))
        speed = 9600; /* use default if property absent */

    /* Read a string property */
    if (of_property_read_string(np, "label", &name))
        name = "unknown";

    dev_info(&pdev->dev, "speed=%u label=%s\n", speed, name);
    return 0;
}
```

Common DT property reading APIs: `of_property_read_u32()`, `of_property_read_string()`, `of_property_read_bool()`, `of_property_read_u32_array()`.

Q24

What isplatform_get_resource()and what resource types can it return?

`platform_get_resource(pdev, type, index)` retrieves a hardware resource described in the device tree (or legacy board data) for a platform device. Resource types include:

`IORESOURCE_MEM` — memory-mapped register range
`IORESOURCE_IRQ` — interrupt number
`IORESOURCE_IO` — I/O port range
`IORESOURCE_DMA` — DMA channel

For IRQs, `platform_get_irq(pdev, 0)` is a more convenient wrapper that also handles error reporting.

Q25

What isioremap()and why is it needed?

Hardware peripherals expose their registers at specific *physical* addresses. Kernel code runs in virtual address space. `ioremap(phys_addr, size)` creates a virtual address mapping for a physical memory-mapped I/O region so the driver can access hardware registers using normal pointer dereferences.

```
void __iomem *base = ioremap(0x40010000, 0x1000);
if (!base)
    return -ENOMEM;

/* Read/write registers */
u32 val = readl(base + REG_STATUS);
writel(0x1, base + REG_CTRL);

/* Always unmap when done */
iounmap(base);
```

> Never directly dereference a physical address in kernel code. Always use **ioremap** and the accessor functions **readl/writel**. Using raw pointer dereference on I/O memory bypasses memory barriers and can cause subtle hardware bugs.

Q26

What is the difference betweenioremap()anddevm_ioremap_resource()?

`ioremap()` is a raw mapping call. You must manually call `iounmap()` in the cleanup path.

`devm_ioremap_resource()` combines `request_mem_region()` (which marks the region as in-use so no other driver can claim it) and `ioremap()` into a single managed call. It is automatically cleaned up when the device is removed. Always prefer this in platform drivers.

Q27

How does driver-device matching work in the platform bus model?

The platform bus core calls `platform_match()` when a new driver or device is registered. It tries four matching strategies in order:

1. OF (Device Tree) match — compares `compatible` string against `driver.of_match_table`. This is the primary method on DT-based systems.
2. ACPI match — for ACPI-based systems.
3. ID table match — compares `platform_device.id_entry` against `driver.id_table`.
4. Name match — compares `platform_device.name` against `driver.name` as a fallback.

Q28

What ismodule_platform_driver()macro and what does it expand to?

`module_platform_driver(my_driver)` is a convenience macro that expands to:

```
static int __init my_driver_init(void)
{
    return platform_driver_register(&my_driver);
}
module_init(my_driver_init);

static void __exit my_driver_exit(void)
{
    platform_driver_unregister(&my_driver);
}
module_exit(my_driver_exit);
```

It eliminates boilerplate init/exit code for drivers that only need to register a platform driver. Similar macros exist for other bus types: `module_i2c_driver()`, `module_spi_driver()`, etc.

Section 4: DMA — Direct Memory Access

DMA (Direct Memory Access) is critical for high-throughput drivers. These questions cover the Linux DMA API, coherent vs. streaming DMA, and IOMMU.

Q29

What is DMA and why is it used in device drivers?

DMA (Direct Memory Access) allows a peripheral device to transfer data directly to/from system memory without CPU involvement. The CPU initiates the transfer, then continues executing other code. The DMA controller signals completion via an interrupt.

DMA is used to achieve high throughput for devices like Ethernet controllers, USB host controllers, storage controllers, audio codecs, and camera interfaces — where CPU-driven byte-by-byte transfers (PIO) would consume most of the CPU's bandwidth.

Q30

What is the DMA Engine API in Linux? Name its key functions.

The **DMA Engine API** (`<linux/dmaengine.h>`) provides a unified interface for all DMA controllers in Linux. Key functions:

`dma_request_channel()` / `dma_request_slave_channel()` — acquire a DMA channel
`dmaengine_slave_config()` — configure the channel (width, burst size, direction)
`dmaengine_prep_slave_single()` / `dmaengine_prep_slave_sg()` — prepare a transfer descriptor
`dmaengine_submit()` — queue the descriptor
`dma_async_issue_pending()` — start execution of queued transfers
`dma_release_channel()` — release the channel when done

Q31

What is the difference between coherent (consistent) DMA and streaming DMA?

Coherent DMA (also called consistent DMA) allocates memory that is always in sync between CPU and device — no explicit cache flushing needed. It is allocated with `dma_alloc_coherent()`. Suitable for long-lived control structures like descriptor rings. More expensive to allocate.

```
void *cpu_addr = dma_alloc_coherent(&pdev->dev,
                                    size,
                                    &dma_handle,
                                    GFP_KERNEL);
/* use dma_handle (bus address) for hardware */
/* use cpu_addr for CPU access */
dma_free_coherent(&pdev->dev, size, cpu_addr, dma_handle);
```

Streaming DMA maps existing kernel buffers for a one-time DMA operation. You must explicitly sync caches before and after the transfer using `dma_map_single()` / `dma_unmap_single()`. More efficient for high-frequency transfers.

```
dma_addr_t dma_handle = dma_map_single(&pdev->dev,
                                       cpu_buf,
                                       size,
                                       DMA_TO_DEVICE);
if (dma_mapping_error(&pdev->dev, dma_handle))
    return -ENOMEM;

/* Initiate DMA transfer using dma_handle */

dma_unmap_single(&pdev->dev, dma_handle, size, DMA_TO_DEVICE);
```

Q32

What is an IOMMU and how does it relate to DMA?

An **IOMMU (Input-Output Memory Management Unit)** sits between the DMA-capable device and system memory. It translates device-visible *I/O virtual addresses* (also called bus addresses or DMA addresses) to actual physical addresses, just as the CPU's MMU translates virtual to physical addresses for the CPU.

Benefits: device isolation (a buggy or malicious device cannot access arbitrary memory), support for 32-bit devices on systems with more than 4GB RAM (SWIOTLB), and scatter-gather support for devices that only understand contiguous buffers.

On Linux, IOMMU support is transparent through the DMA mapping API — the same `dma_map_single()` / `dma_alloc_coherent()` calls work whether an IOMMU is present or not.

Q33

What is a scatter-gather (SG) list and when is it used?

A **scatter-gather list** is an array of `struct scatterlist` entries, each describing a physically non-contiguous memory segment. It allows a DMA transfer to read from or write to multiple discontiguous buffer fragments as if they were a single contiguous stream.

SG lists are used in storage drivers (reading a file that is fragmented in physical memory), network drivers (chaining socket buffers), and video drivers.

```
struct scatterlist sg[3];
sg_init_table(sg, 3);
sg_set_buf(&sg[0], buf0, len0);
sg_set_buf(&sg[1], buf1, len1);
sg_set_buf(&sg[2], buf2, len2);

int nents = dma_map_sg(&pdev->dev, sg, 3, DMA_TO_DEVICE);
/* Pass nents and sg to DMA engine */
dma_unmap_sg(&pdev->dev, sg, 3, DMA_TO_DEVICE);
```

Q34

What is SWIOTLB and when does Linux use it?

SWIOTLB (Software I/O Translation Lookaside Buffer) is a software bounce-buffer mechanism Linux uses when a DMA-capable device cannot address all of physical memory (e.g., a 32-bit DMA device on a machine with >4GB RAM, or when no IOMMU is present).

Linux copies data to/from a special low-memory bounce buffer that the device can reach, performs the DMA, then copies back. This is transparent to the driver — the DMA mapping API handles it automatically. It comes at a performance cost due to the extra copy.

Q35

What doesdma_set_mask_and_coherent()do?

It informs the kernel how many bits of physical address the device's DMA engine can use. This allows the kernel to decide whether to use direct DMA or bounce buffers.

```
/* In probe(): declare device supports 64-bit DMA */
if (dma_set_mask_and_coherent(&pdev->dev, DMA_BIT_MASK(64))) {
    /* 64-bit failed, try 32-bit */
    if (dma_set_mask_and_coherent(&pdev->dev, DMA_BIT_MASK(32))) {
        dev_err(&pdev->dev, "No usable DMA configuration\n");
        return -ENODEV;
    }
}
```

Always call this in `probe()` before performing any DMA operations.

Q36

What is a DMA descriptor ring (descriptor queue)?

A **DMA descriptor ring** is a circular queue of DMA descriptors in coherent memory, shared between the driver (CPU) and the DMA controller (hardware). Each descriptor contains: a buffer address, buffer length, control flags (interrupt on completion, end of packet, etc.), and a pointer to the next descriptor.

The CPU populates descriptors and moves the *tail pointer* forward to signal new work. The hardware reads descriptors, performs transfers, and moves the *head pointer* forward. This is the standard pattern in Ethernet (e.g., dwmac), NVMe, and SATA controllers.

Q37

Explain cache coherency issues in DMA.

On systems without hardware cache coherency (common on ARM SoCs), the CPU cache and main memory can be out of sync:

CPU writes → DMA reads (DMA_TO_DEVICE): The CPU must *flush* its cache lines to memory before starting DMA, so the device sees up-to-date data.

DMA writes → CPU reads (DMA_FROM_DEVICE): After DMA completes, the CPU must *invalidate* its cache lines so it reads the device's freshly written data and not stale cached values.

The Linux DMA API handles this automatically when you use `dma_map_single()` / `dma_unmap_single()` with the correct direction flags (`DMA_TO_DEVICE`, `DMA_FROM_DEVICE`, `DMA_BIDIRECTIONAL`). Coherent DMA buffers (`dma_alloc_coherent`) are mapped uncached, so no explicit flushing is needed.

Q38

Write a simple DMA Engine slave transfer example.

```
#include <linux/dmaengine.h>
#include <linux/dma-mapping.h>

static struct dma_chan *dma_chan;
static DECLARE_COMPLETION(dma_done);

static void dma_callback(void *data)
{
    complete(&dma_done); /* signal transfer complete */
}

static int do_dma_transfer(struct device *dev,
                           void *src, dma_addr_t dst_phys, size_t len)
{
    struct dma_async_tx_descriptor *desc;
    dma_addr_t src_dma;

    src_dma = dma_map_single(dev, src, len, DMA_TO_DEVICE);
    if (dma_mapping_error(dev, src_dma))
        return -ENOMEM;

    desc = dmaengine_prep_slave_single(dma_chan, src_dma, len,
                                       DMA_MEM_TO_DEV,
                                       DMA_PREP_INTERRUPT);
    if (!desc) {
        dma_unmap_single(dev, src_dma, len, DMA_TO_DEVICE);
        return -ENOMEM;
    }

    desc->callback       = dma_callback;
    desc->callback_param = NULL;
    init_completion(&dma_done);

    dmaengine_submit(desc);
    dma_async_issue_pending(dma_chan);

    wait_for_completion(&dma_done); /* wait for IRQ callback */
```

```
    dma_unmap_single(dev, src_dma, len, DMA_TO_DEVICE);
    return 0;
}
```

Section 5: Interrupts & Bottom Halves

Interrupt handling is one of the most timing-sensitive parts of a driver. These questions cover top/bottom halves, softirqs, tasklets, and workqueues.

Q39

What is the difference between the top half and bottom half of interrupt handling?

The **top half** (hard IRQ handler) runs immediately when an interrupt fires. It executes with interrupts disabled on the local CPU and must be extremely fast. Its job is to acknowledge the interrupt, save essential data, and schedule deferred work.

The **bottom half** is deferred processing that runs later with interrupts enabled. Linux provides several mechanisms: **softirqs** (fixed set, runs in interrupt context), **tasklets** (built on softirqs, per-CPU, serialised), and **workqueues** (runs in process context, can sleep).

Q40

How do you request an interrupt in a driver? What flags are important?

```
#include <linux/interrupt.h>

static irqreturn_t my_isr(int irq, void *dev_id)
{
    struct my_dev *dev = dev_id;
    u32 status = readl(dev->base + REG_STATUS);
    if (!(status & MY_INT_BIT))
        return IRQ_NONE; /* not our interrupt */

    writel(status, dev->base + REG_STATUS); /* clear interrupt */
    tasklet_schedule(&dev->tasklet);
    return IRQ_HANDLED;
}

/* In probe(): */
ret = devm_request_irq(&pdev->dev, irq, my_isr,
                       IRQF_SHARED, "my_driver", dev);
```

Key flags: `IRQF_SHARED` — allows multiple devices to share one IRQ line; `IRQF_TRIGGER_RISING` / `IRQF_TRIGGER_LEVEL_HIGH` — edge vs level triggering; `IRQF_ONESHOT` — used with threaded IRQs to keep the line disabled until the thread completes.

Q41

What is a threaded IRQ and when should you use it?

A **threaded IRQ** (introduced in Linux 2.6.30) splits interrupt handling into a minimal hard IRQ handler (top half) and a kernel thread that runs the bulk of the processing. The thread can sleep, take mutexes, and call blocking APIs — things forbidden in a hard IRQ context.

```
static irqreturn_t my_hard_isr(int irq, void *data)
{
    /* Minimal: just acknowledge */
    return IRQ_WAKE_THREAD; /* wake the thread */
}

static irqreturn_t my_thread_fn(int irq, void *data)
{
    struct my_dev *dev = data;
    /* Can sleep, take mutex, do I/O */
    mutex_lock(&dev->lock);
    process_data(dev);
    mutex_unlock(&dev->lock);
    return IRQ_HANDLED;
}

devm_request_threaded_irq(&pdev->dev, irq,
                          my_hard_isr,   /* top half    */
                          my_thread_fn,  /* bottom half */
                          IRQF_ONESHOT,
                          "my_driver", dev);
```

Use threaded IRQs when your interrupt handler needs to do significant work or call blocking APIs.

Q42

What is the difference between a tasklet and a workqueue?

Tasklet: Runs in *softirq context* (interrupt context). Cannot sleep. Cannot call blocking functions. Executes on the CPU that scheduled it. Fast, but limited in what it can do.

Workqueue: Runs in *process context* (kernel thread). Can sleep. Can call blocking functions, take mutexes, allocate memory with `GFP_KERNEL`. Slower to schedule but much more flexible.

> In modern kernels, tasklets are deprecated in favour of threaded IRQs or workqueues. The kernel documentation now recommends avoiding tasklets in new code.

Q43

Can you callprintk()orkmalloc()from an interrupt handler?

`printk()` — Yes, it is safe to call from interrupt context, though it may be slow. In practice you should avoid excessive printing in ISRs.

`kmalloc(GFP_ATOMIC)` — Yes, with the `GFP_ATOMIC` flag, which tells the allocator not to sleep. It may fail if no memory is immediately available.

`kmalloc(GFP_KERNEL)` — **No**. This can sleep, which is forbidden in interrupt context. Using it from an ISR will cause a kernel oops or deadlock.

Never call any function that can sleep from an interrupt handler or softirq context. Use **GFP_ATOMIC** for memory allocations in ISRs.

Q44

How do you check if you are in interrupt context in kernel code?

```
#include <linux/hardirq.h>

if (in_interrupt()) {
    /* running in hard IRQ or softirq context — cannot sleep */
}

if (in_atomic()) {
    /* cannot sleep: interrupt context, spinlock held, or RCU read-side */
}

if (in_task()) {
    /* running in process context — can sleep */
}
```

These macros check flags in the thread info / preempt count. Knowing your execution context is crucial before calling any function that might sleep.

Section 6: Synchronization, Locking & Memory

Synchronization and memory management are where subtle bugs hide. These questions cover spinlocks, mutexes, RCU, memory barriers, and GFP flags.

Q45

What is the difference between a spinlock and a mutex in the kernel?

Spinlock: The contending CPU *busy-waits* in a loop until the lock is released. It does not sleep. Can be used in interrupt context. Very low overhead for short critical sections. Disables preemption on the local CPU while held.

Mutex: The contending task *sleeps* until the lock is available. Cannot be used in interrupt context or with interrupts disabled. Allows the CPU to do useful work while waiting. Suited for long critical sections.

> **Rule of thumb:** Use a spinlock if you need the lock from interrupt context or if the critical section is very short (a few instructions). Use a mutex everywhere else.

Q46

What isspin_lock_irqsave()and when do you need it?

`spin_lock_irqsave(&lock, flags)` acquires the spinlock AND disables interrupts on the local CPU, saving the previous interrupt state in `flags`. `spin_unlock_irqrestore(&lock, flags)` releases the lock and restores the saved interrupt state.

You need this when a resource is shared between an interrupt handler and process context code. Without disabling interrupts, an interrupt could fire while the spinlock is held in process context, the ISR could try to take the same spinlock, and deadlock.

```
unsigned long flags;
spin_lock_irqsave(&dev->lock, flags);
/* critical section — safe from both process and IRQ context */
dev->count++;
spin_unlock_irqrestore(&dev->lock, flags);
```

Q47

What is RCU (Read-Copy-Update) and what is its primary use case?

RCU is a synchronization mechanism that allows reads to proceed concurrently with writes without any locking on the read side. Reads are extremely cheap (a memory barrier). Writers create a new copy of the data, atomically swap in the pointer, then wait for all existing readers to finish before freeing the old copy.

RCU is used for read-mostly data structures like routing tables, network device lists, module lists, and file system dentries. It is one of the key reasons Linux scales well on many-core systems.

```
/* Reader: */
rcu_read_lock();
p = rcu_dereference(global_ptr);
```

```
if (p) use(p);
rcu_read_unlock();

/* Writer: */
new_p = kmalloc(...);
old_p = rcu_dereference(global_ptr);
rcu_assign_pointer(global_ptr, new_p);
synchronize_rcu(); /* wait for readers of old_p */
kfree(old_p);
```

Q48

What are memory barriers and why are they needed in drivers?

Modern CPUs and compilers reorder memory operations for performance. In device drivers, incorrect ordering between register writes or between DMA and CPU memory accesses can cause hardware to see operations in the wrong order, leading to silent data corruption or hardware hangs.

Linux provides memory barrier macros to enforce ordering:

`mb()` — full memory barrier (all loads/stores ordered)
`rmb()` — read memory barrier
`wmb()` — write memory barrier
`smp_mb()` — SMP-safe full barrier
`dma_wmb()` / `dma_rmb()` — DMA-specific barriers

The I/O accessor functions `readl()`/`writel()` already include implicit barriers for MMIO — but you still need explicit barriers when ordering DMA descriptor updates.

Q49

What is the difference betweenreadl()/writel()and direct pointer dereference for MMIO?

`readl()` and `writel()` are accessor macros for memory-mapped I/O that:

1. Prevent the compiler from caching or reordering the access (act as a compiler barrier)
2. Include the necessary CPU memory barriers for I/O ordering
3. Handle endianness correctly (`readl_relaxed()` skips the CPU barrier for performance when ordering is guaranteed by context)

Direct pointer dereference of MMIO memory — even through a `volatile` pointer — is not safe because it lacks the memory ordering guarantees. The kernel coding style mandates `readl()`/`writel()` for all MMIO accesses.

Q50

How do you debug a kernel oops or panic caused by a driver?

A kernel oops prints the faulting instruction's address, register dump, and a call trace to the kernel log. The standard debugging workflow:

1. Decode the oops — use `addr2line -e vmlinux [address]` or the `scripts/decode_stacktrace.sh` script to convert addresses to source file/line numbers.
2. Reproduce under KASAN — compile with `CONFIG_KASAN=y` (Kernel Address Sanitizer) to catch out-of-bounds and use-after-free.
3. Dynamic debug — enable `pr_debug()` output with `dyndbg`.
4. KGDB / QEMU — attach a kernel debugger via `kgdb` or run the kernel in QEMU with GDB for interactive debugging.
5. ftrace — use `CONFIG_FTRACE` to trace function calls and identify where execution goes wrong.
6. lockdep — enable `CONFIG_LOCKDEP` to detect locking order violations and potential deadlocks before they happen.

Quick Revision Cheatsheet

Character driver flow: alloc_chrdev_region → cdev_init → cdev_add → class_create → device_create → implement file_operations

Platform driver flow: define of_match_table → implement probe/remove → register with module_platform_driver()

DMA flow (streaming): dma_set_mask → dma_map_single → initiate transfer → wait for IRQ → dma_unmap_single

Locking rule: mutex in process context; spinlock when used from IRQ; spin_lock_irqsave when shared with ISR

Memory rule: kmalloc(GFP_KERNEL) in process context; kmalloc(GFP_ATOMIC) in IRQ; vmalloc for large non-DMA buffers

Ready to Crack Your Embedded Linux Interview?

Join the EmbeddedShiksha Interview Prep Bootcamp — a structured program covering C & DSA for Embedded, RTOS, Linux Kernel & Drivers, and Mock Interviews. Everything you need to land a top-tier embedded systems role.

✓ C Programming & DSA for Embedded Systems
✓ RTOS — FreeRTOS, task scheduling, queues, semaphores
✓ Linux Kernel Internals & Device Drivers
✓ Live Mock Interviews with expert feedback

Enroll Now → https://www.embeddedshiksha.com/courses/Embedded-System-Interview-Prep-Bootcamp-30-6975062bb8c8c77d7496a98b

Share this guide with a friend preparing for interviews!
Follow EmbeddedShiksha on LinkedIn for more kernel & embedded content.